

Phase 5: Infrastructure as Code (IaC) and Secrets Management

5.1 Terraform Installation and Configuration

Terraform was installed and configured on the development environment. The installation was verified successfully using the version command.

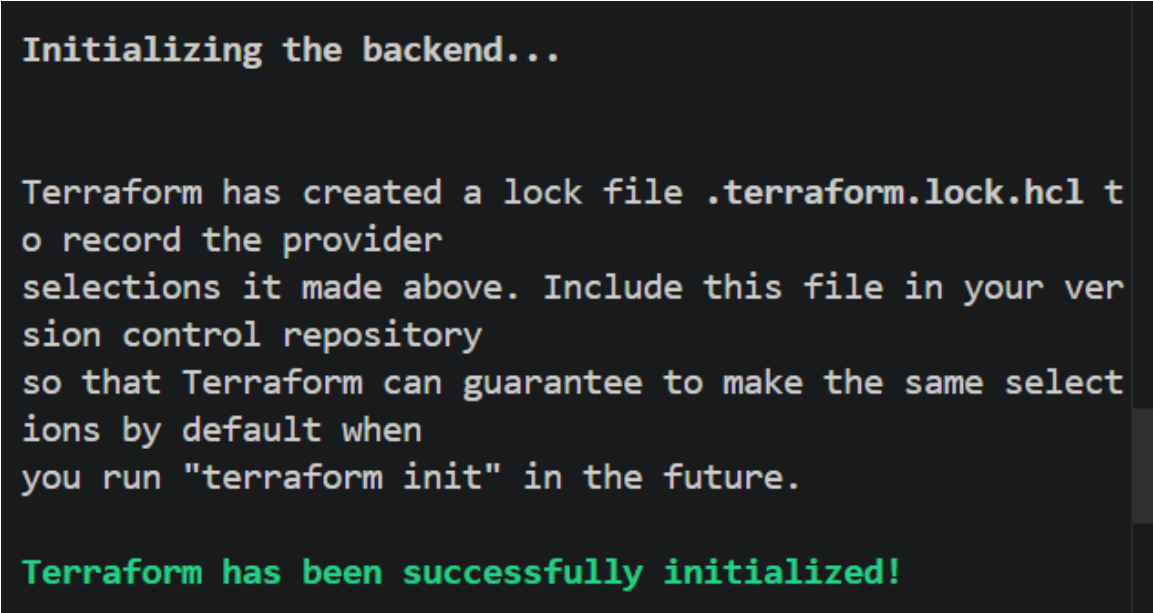


```
Terraform v1.15.6  
on windows_amd64
```

Figure 5.1: Terraform Installation Verification

5.2 Terraform Infrastructure Initialization

A Terraform configuration file was created to define infrastructure resources. The initialization process downloaded and configured the required provider plugins necessary for resource provisioning.



```
Initializing the backend...
```

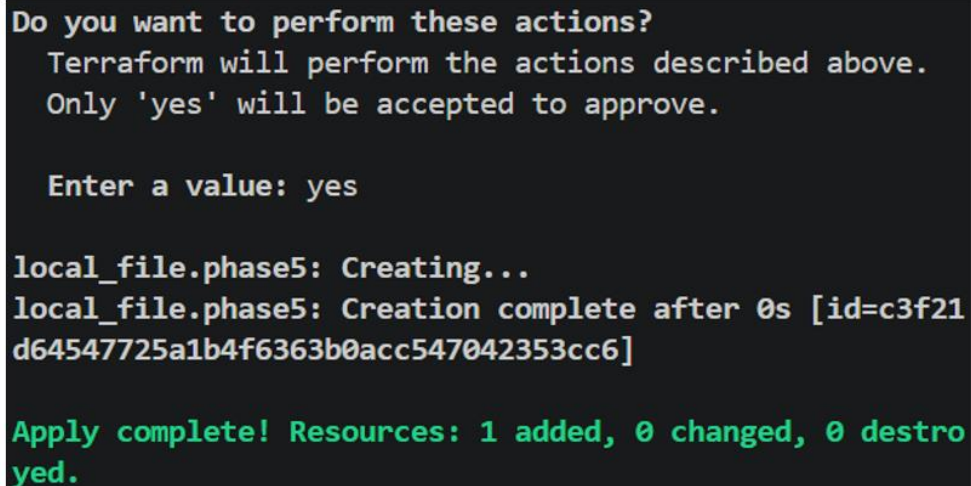
```
Terraform has created a lock file .terraform.lock.hcl to  
record the provider selections it made above. Include this  
file in your version control repository so that Terraform  
can guarantee to make the same selections by default when  
you run "terraform init" in the future.
```

```
Terraform has been successfully initialized!
```

Figure 5.2: Terraform Initialization

5.3 Infrastructure Provisioning using Terraform

Terraform was used to provision a sample infrastructure resource through Infrastructure as Code principles. The configuration was validated and deployed successfully.

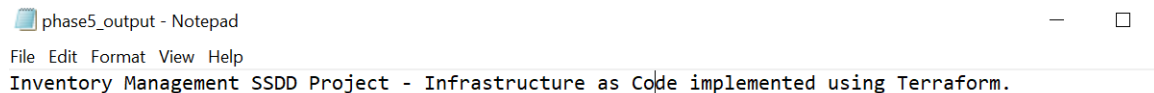
A terminal window with a dark background and light-colored text. The text shows the Terraform CLI prompts and output for a deployment. It starts with a confirmation prompt, followed by the user input 'yes', and then shows the successful creation of a resource with its ID. The final line indicates the application is complete with resource counts.

```
Do you want to perform these actions?  
Terraform will perform the actions described above.  
Only 'yes' will be accepted to approve.  
  
Enter a value: yes  
  
local_file.phase5: Creating...  
local_file.phase5: Creation complete after 0s [id=c3f21  
d64547725a1b4f6363b0acc547042353cc6]  
  
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Figure 5.3: Terraform Deployment Execution

5.4 Generated Infrastructure Resource

After deployment, Terraform automatically generated the specified output resource, demonstrating successful automation and infrastructure provisioning.

A screenshot of a Notepad window titled 'phase5_output - Notepad'. The window has a menu bar with 'File', 'Edit', 'Format', 'View', and 'Help'. The main text area contains the output of the Terraform deployment: 'Inventory Management SSDD Project - Infrastructure as Code implemented using Terraform.'

phase5_output - Notepad

File Edit Format View Help

Inventory Management SSDD Project - Infrastructure as Code implemented using Terraform.

Figure 5.4: Terraform Generated Resource

5.5 HashiCorp Vault Installation

HashiCorp Vault was installed to provide centralized and secure storage for application secrets, credentials, and sensitive configuration data.

```
Vault v2.0.2 (a71d5add7ef4111ec4015e9a81b06388f652e2ac)  
, built 2026-06-04T13:18:11Z
```

Figure 5.5: Vault Installation Verification

5.6 Vault Authentication

Authentication was performed using the generated root token provided by the Vault development server.

```
Success! You are now authenticated. The token informati  
on displayed below  
is already stored in the token helper. You do NOT need  
to run "vault login"  
again. Future Vault requests will automatically use thi  
s token.
```

Key	Value
---	-----
token	hvs.6E5XFNQ3jbtdwu88yn88fwlo
token_accessor	qAiVneG2XKwDWRWmYG4ne7Ys
token_duration	∞

Figure 5.6: Vault Authentication

5.7 Secure Secret Storage

Application secrets including database credentials, JWT secrets, and MongoDB connection strings were securely stored within Vault's Key-Value secret engine.

```
Inventory
==== Secret Path ====
secret/data/inventory

===== Metadata =====
Key                Value
---              -
created_time       2026-06-11T14:56:55.7728643Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

===== Data =====
Key                Value
---              -
db_password        inventory123
jwt_secret          inventory_jwt_key
mongo_uri           mongodb://localhost:27017/inventory
```

Figure 5.7: Secure Secret Storage and Retrieval

Phase 6: Monitoring, DevSecOps, and AI-Based Security

6.1 AI-Based Anomaly Detection

A lightweight anomaly detection prototype was implemented to identify unusual system behavior. The model analyzes CPU utilization values and identifies abnormal resource consumption patterns that may indicate malicious activity or performance degradation.

```
anomaly_detection.py
AI/ML Anomaly Detection for Inventory Management System
-----
Average CPU Usage: 27.50%

CPU Usage: 12% => Normal
CPU Usage: 15% => Normal
CPU Usage: 14% => Normal
CPU Usage: 16% => Normal
CPU Usage: 13% => Normal
CPU Usage: 95% => Anomaly Detected
```

Figure 6.1: AI-Based Anomaly Detection Prototype

6.2 System Monitoring Prototype

A monitoring prototype was developed to continuously observe CPU and memory utilization. Alerts are generated whenever predefined resource thresholds are exceeded.

```
tem_monitor.py
Inventory Management System Monitoring
-----
Check 1: CPU Usage = 35% | Memory Usage = 71%
Check 2: CPU Usage = 44% | Memory Usage = 25%
Check 3: CPU Usage = 12% | Memory Usage = 21%
Check 4: CPU Usage = 68% | Memory Usage = 80%
Alert: High resource usage detected!
Check 5: CPU Usage = 32% | Memory Usage = 56%
```

Figure 6.2: System Monitoring Implementation

6.3 DevSecOps CI/CD Automation

A GitHub Actions workflow was developed to automate security and monitoring activities. The workflow executes anomaly detection and monitoring scripts automatically during software delivery operations.

```
name: Phase 6 DevSecOps Pipeline

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  security-and-monitoring-check:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Run AI Anomaly Detection Script
        run: python monitor/ai/anomaly_detection.py

      - name: Run Monitoring Prototype
        run: python monitor/system_monitor.py

      - name: DevSecOps Pipeline Completed
        run: echo "Phase 6 CI/CD security pipeline completed successfully."
```

Figure 6.3: DevSecOps CI/CD Workflow Configuration

6.4 Proposed Security Monitoring Architecture

The proposed monitoring architecture integrates application monitoring, anomaly detection, centralized log collection, and security event management capabilities. Security events generated by monitoring components can be forwarded to enterprise-grade SIEM solutions such as IBM QRadar for further analysis and incident response.

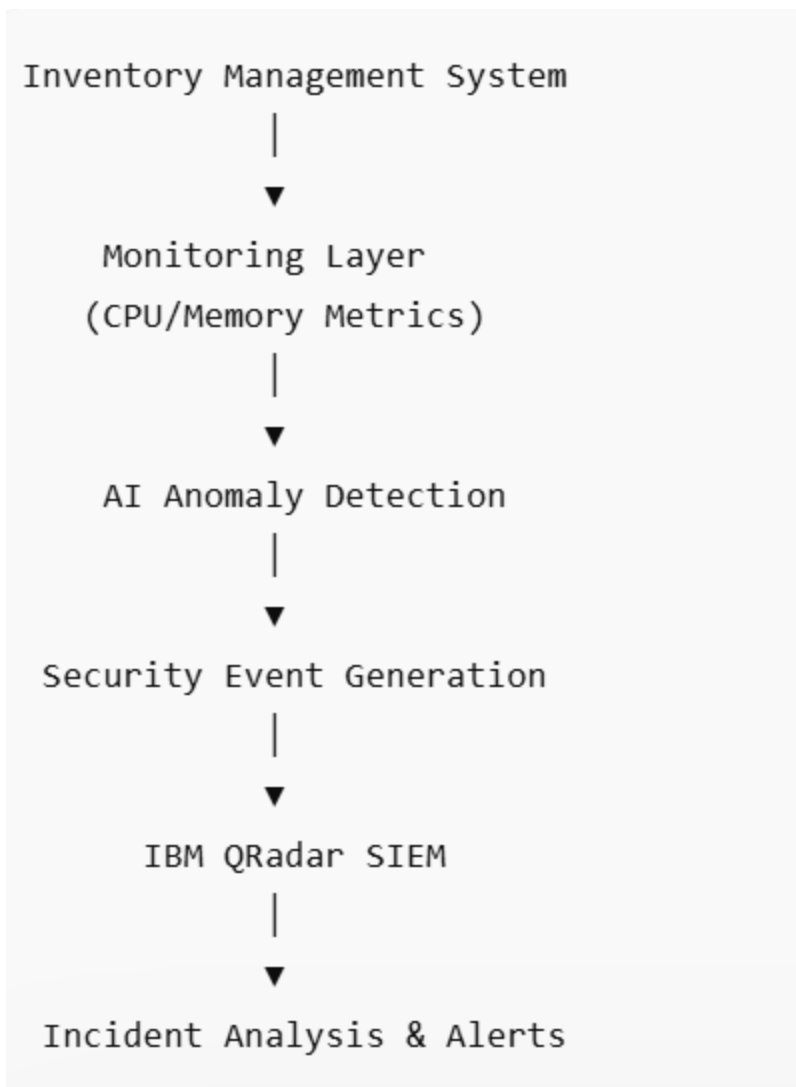


Figure 6.4: Security Monitoring and SIEM Integration Architecture

Conclusion

Phase 5 successfully demonstrated Infrastructure as Code implementation using Terraform and secure secret management using HashiCorp Vault. Phase 6 introduced monitoring, DevSecOps automation, and AI-assisted anomaly detection capabilities. Together, these implementations improve deployment consistency, enhance security, reduce operational risks, and provide greater visibility into system behavior. The completed work contributes to a more secure and manageable Inventory Management System aligned with modern DevSecOps and cloud-native security practices.